

DOCUMENTATION WEEK 4

MONDAY

morning

- mid-term presentations
- peer feedback:
 - aesthetically pleasing
 - add more variation / variables -> must be interesting for 5 minutes

afternoon

- feedback / input by Stefanie & Yann
 - Which part of the music (amplitude, etc.) should influence the visuals? Why?
 - Which words are displayed? Why?
 - What story do we want to tell? Or is it rather an aesthetic exploration?
 - History of fonts
 - use old computer fonts
 - development of fonts: hand-cut -> lead letters -> computer fonts -> digital fonts not limited by physical constraints
 - How do you want to fill the 5 minutes of performance -> sequence, motion, change between snippets
 - source of inspiration: Ted Davis, Remi

TUESDAY

excursion to Basel

Due to my work shift in the evening, I unfortunately could not participate in the excursion. Instead, I experimented further with the coding:

TEXT TRACE

```
let word1 = "SOUND"
let word2 = "*"
let word3 = "---"
let spac = 10
let x1 = 0
let y1 = 0
let x2 = 0
let y2 = 0
let x3 = 0
let y3 = 0

function setup() {
  createCanvas(windowWidth, windowHeight)
  background(0)
}

function draw() {
  textSize(10)
  spac = abs(sin(frameCount * 0.02) * 30)
  x1 = sin(frameCount * 0.04) * 500
  y1 = cos(frameCount * 0.01) * 400
  fill(0,255,0)
  textSpacing(word1, spac, width/2 + x1, height/2 + y1)

  textSize(20)
  fill(255,0,255)
  x2 = sin(frameCount * 0.04) * 800
  y2 = sin(frameCount * 0.08) * 200
  textSpacing(word2, spac,width/2 + x2, height/2 + y2)

  textSize(20)
  fill(0,0,255)
```

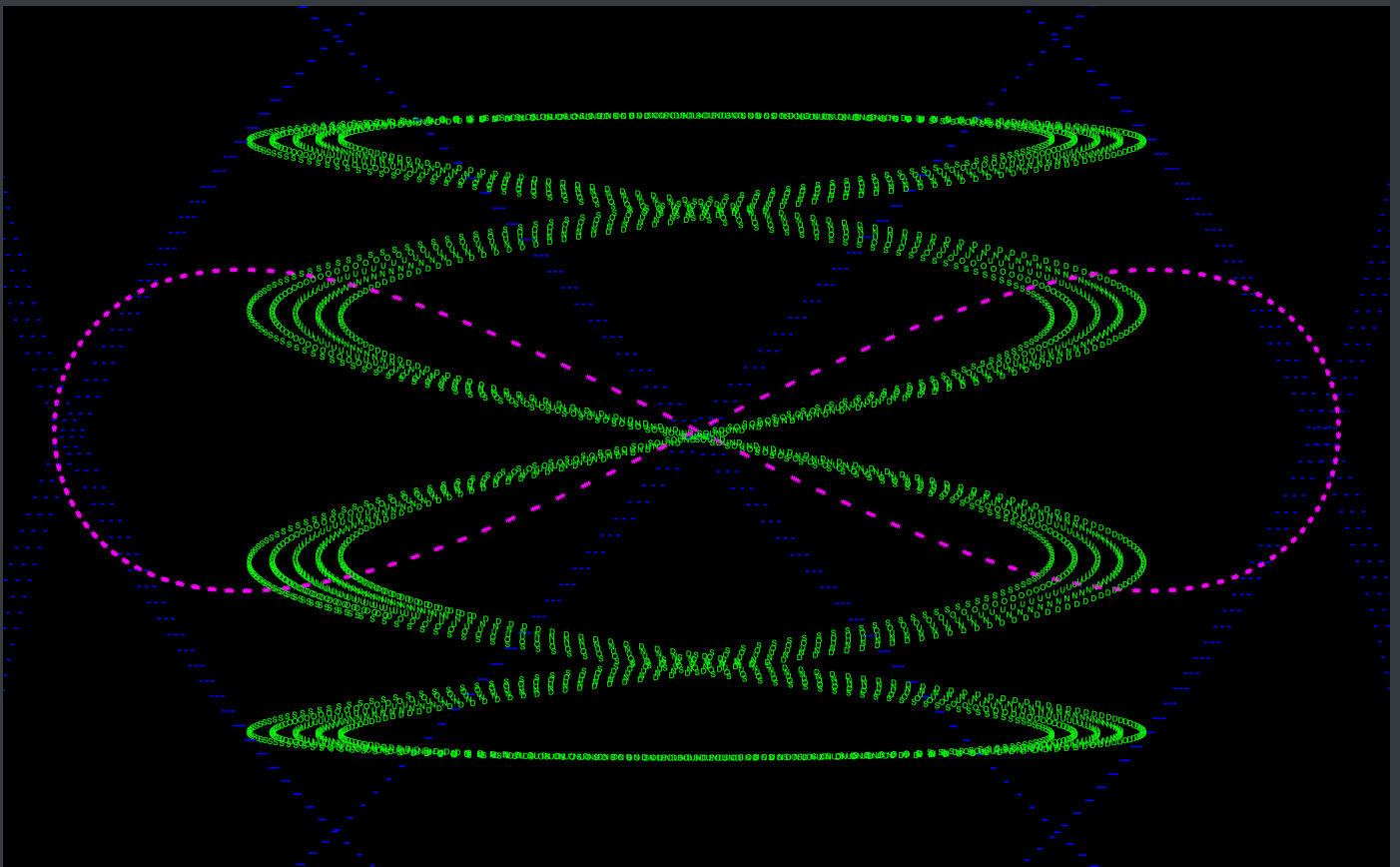
```

x3 = sin(frameCount * 0.02) * 900
y3 = cos(frameCount * 0.03) * 700
textSpacing(word3, spac -20 ,width/2 + x3, height/2 + y3)

}

function textSpacing(txt, spacing, x, y) {
  let totalWidth = 0;
  for(let char of txt) {
    totalWidth += textWidth(char) + spacing;
  }
  totalWidth -= spacing;
  x -= totalWidth / 2;
  for(let char of txt) {
    text(char, x, y);
    x += textWidth(char) + spacing;
  }
}

```



DISTORTED NUMBER REPETITION

```
let amountX = [1, 2, 2, 3, 3, 5, 9, 20, 40, 80]
let amountY = [1, 1, 2, 3, 5, 5, 9, 15, 35, 60]
let wordArray = ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]

function setup() {
  createCanvas(windowWidth, windowHeight)
  frameRate(2)
}

function draw() {
  background(255, 0, 0)
  textAlign(LEFT, TOP)

  // i always counts from 1 until end of amountX Array
  let i = frameCount % amountX.length

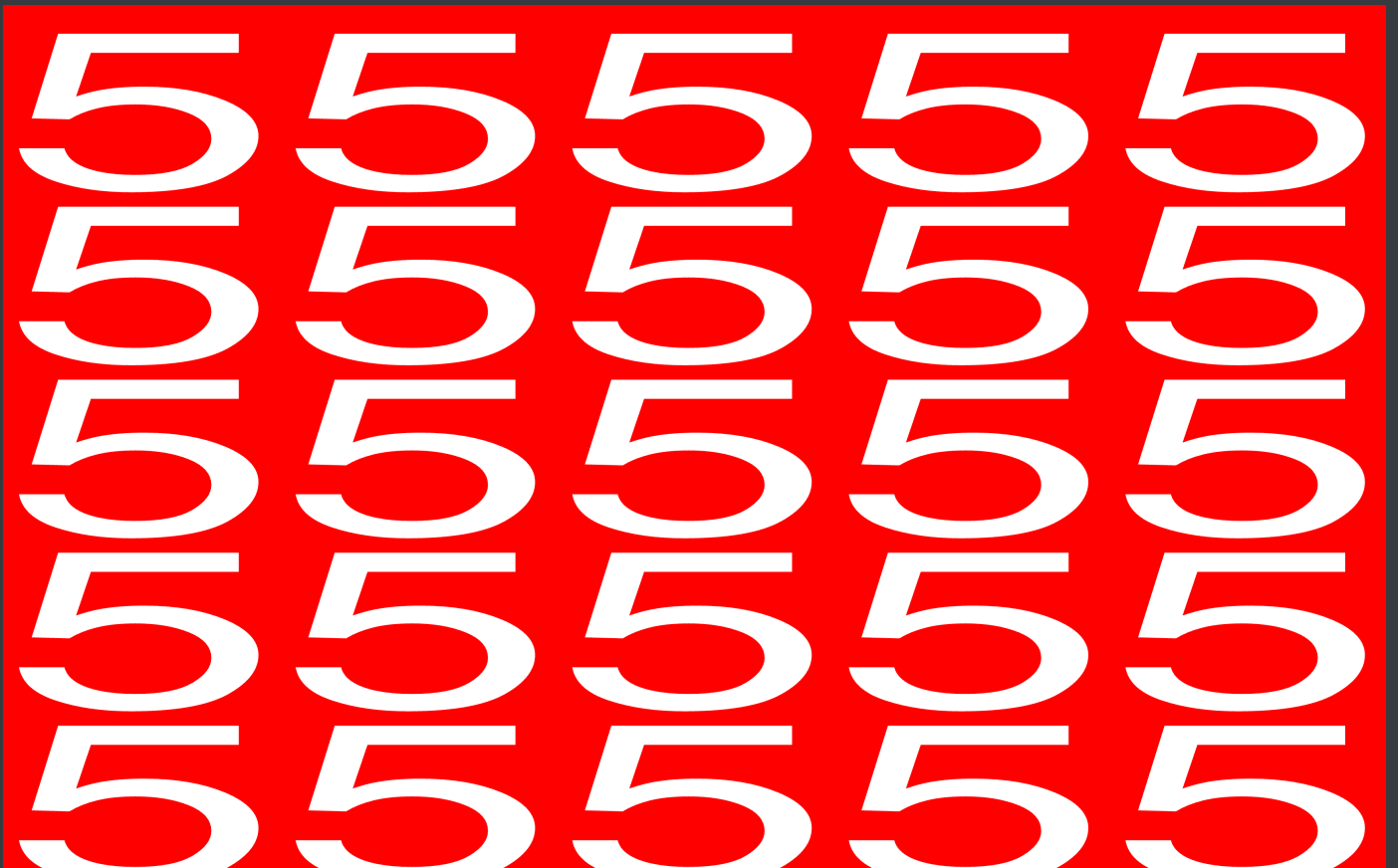
  // in different arrays, element i is used
  let rows = amountY[i]
  let cols = amountX[i]
  let word = wordArray[i]

  // writing of horizontally repeated text is repeated vertically
  // with for-loop -> for every run, posY changes = duplication
  for(let j = 0; j < rows; j++) {
    textSize((height / rows) * 1.3)
    // height/rows = line height
    // vertical position is j * line height -> lines are drawn
    // beneath and not on top of each other
    let posY = j * (height / rows)
    // text is repeated horizontally as often as array for cols says
    let repeatedText = word.repeat(cols)
    // horizontal scale factor is calculated so that whole text fills
    // width of screen
    let scaleX = width / textWidth(repeatedText)

    fill(255)
    push()
```

```
// text is distorted according to scale factor
scale(scaleX, 1)
// repeated text is written on the posY which changes with every run
text(repeatedText, 0, posY)
pop()
}

}
```



RANDOM CHARACTER FILL

```
let characters = ["-", "0"]
let posX = 0
let posY = 0
let myFont

function preload() {
  myFont = loadFont('/data/spacemono.ttf')
}
```

```

function setup() {
  createCanvas(windowWidth, windowHeight)
  background(0)
  frameRate(5)
}

function draw() {
  textFont(myFont)
  textAlign(LEFT, TOP)
  textSize(32)
  fill(255)

  // j counts from 1 to the max of elements in characters
  let j = frameCount % characters.length

  // chooses random number between 1 and 8, multiplied with 3
  let randomRep = 5 * floor(random(1, 8))
  // the letter on the j position in the characters array is chosen
  // and repeated as many times as the randomRep chooses
  let charRep = characters[j].repeat(randomRep)

  // text is written
  text(charRep, posX, posY)
  // offset of pos of next letter group to avoid overlay
  posX += textWidth(charRep)

  // if line is full, it will move to the next line on the right
  if(posX >= width) {
    posX = 0
    posY += textSize()
  }

  // if screen is full, the background is repainted over it all = clear
  // and the position is set back to 0,0
  if(posY > height) {
    background(0)
    posX = 0
    posY = 0
  }
}

```



```
// speed changes according to sin; off-set of different walkers with "+..."
let speed1 = abs(sin((frameCount + 0) * 0.04) * 10)
let speed2 = abs(sin((frameCount + 20) * 0.02) * 10)
let speed3 = abs(sin((frameCount + 40) * 0.04) * 10)
let speed4 = abs(sin((frameCount + 60) * 0.03) * 10)

// WALKER 1
// text is written, posX is altered, text is rewritten, etc.
text(word, x1, y1)
x1 += speed1
// if walker reaches right window border, new walker on left
// border on a random line on y-axis starts
if (x1 >= width) {
  x1 = 0
  y1 = floor(random(0, height / textSize() - 1)) * textSize()
}

// WALKER 2
text(word, x2, y2)
x2 += speed2
if (x2 >= width) {
  x2 = 0
  y2 = floor(random(0, height / textSize() - 1)) * textSize()
}

// WALKER 3
text(word, x3, y3)
x3 += speed3
if (x3 >= width) {
  x3 = 0
  y3 = floor(random(0, height / textSize() - 1)) * textSize()
}

// WALKER 4
text(word, x4, y4)
x4 += speed4
if (x4 >= width) {
  x4 = 0
  y4 = floor(random(0, height / textSize() - 1)) * textSize()
}
```



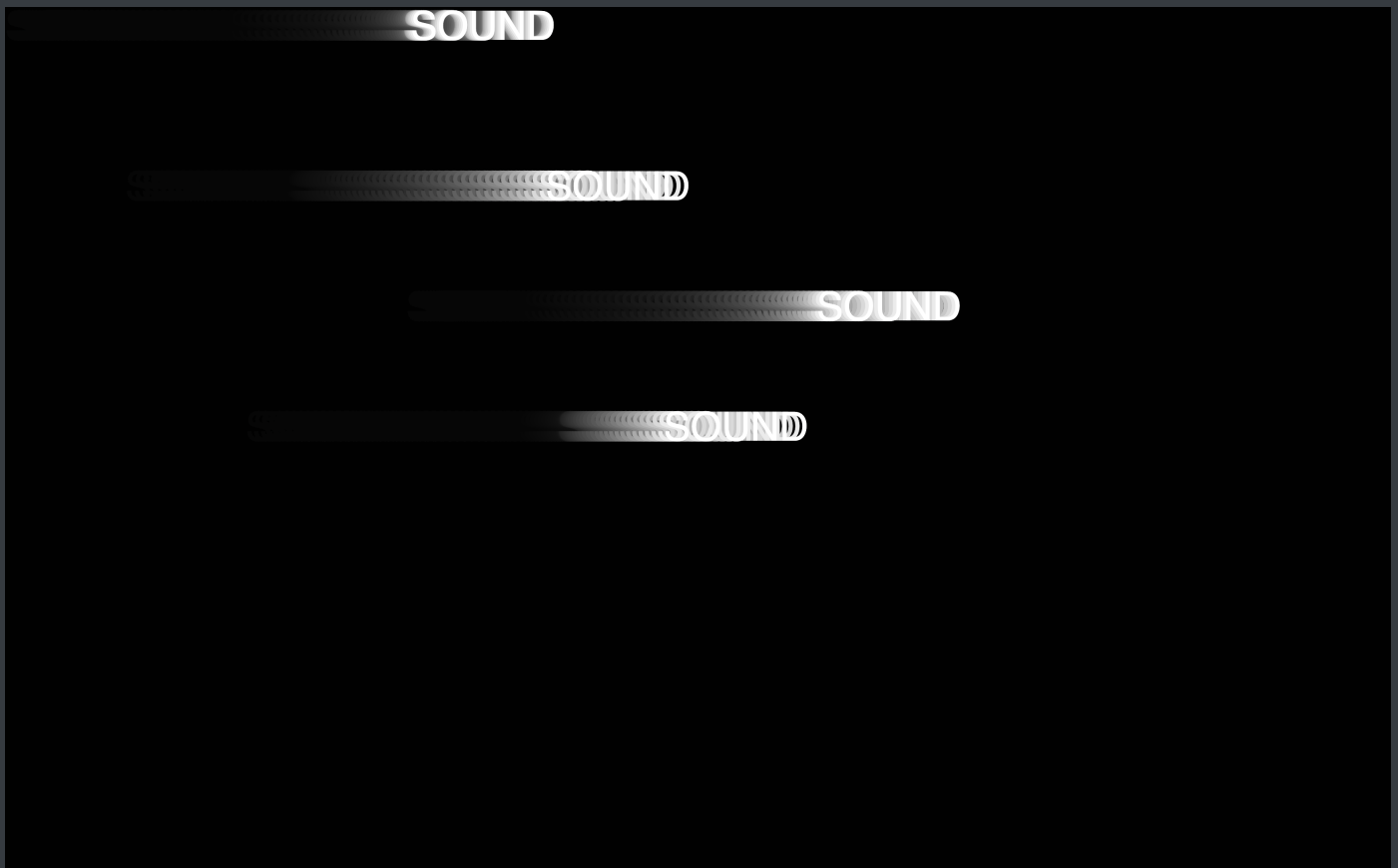
```

}

// function textWalker(word, posX, posY) {
//   text(word, posX, posY)
//   posX += 10

//   if(posX>=width){
//     posX=0
//     let randY = floor(random(0, (height / textSize() - 1)))
//     posY=randY*textSize()
//   }
// }

```



MANY WORD WALKERS

```

let word = "SOUND"
let size = 80

```

```
let x0 = 1100,  
    y0 = 0 * size  
let x1 = 100,  
    y1 = 1 * size  
let x2 = 500,  
    y2 = 2 * size  
let x3 = 300,  
    y3 = 3 * size  
let x4 = 900,  
    y4 = 4 * size  
let x5 = 1300,  
    y5 = 5 * size  
let x6 = 600,  
    y6 = 6 * size  
let x7 = 0,  
    y7 = 7 * size  
let x8 = 800,  
    y8 = 8 * size  
let x9 = 100,  
    y9 = 9 * size  
let x10 = 1500,  
    y10 = 10 * size  
let x11 = 700,  
    y11 = 11 * size  
let x12 = 200,  
    y12 = 12 * size  
let x13 = 900,  
    y13 = 13 * size  
let x14 = 400,  
    y14 = 14 * size  
  
function setup() {  
  createCanvas(windowWidth, windowHeight)  
  frameRate(15)  
}  
  
function draw() {  
  background(0, 5)  
  textAlign(LEFT, TOP)  
  textSize(size)
```

```
stroke(255)
fill(0)
//fill(255)
let speed0 = abs(sin((frameCount + 0) * 0.03) * 10)
let speed1 = abs(sin((frameCount + 20) * 0.05) * 10)
let speed2 = abs(sin((frameCount + 40) * 0.04) * 10)
let speed3 = abs(sin((frameCount + 60) * 0.04) * 10)
let speed4 = abs(sin((frameCount + 80) * 0.02) * 10)
let speed5 = abs(sin((frameCount + 100) * 0.03) * 10)
let speed6 = abs(sin((frameCount + 120) * 0.04) * 10)
let speed7 = abs(sin((frameCount + 140) * 0.05) * 10)
let speed8 = abs(sin((frameCount + 160) * 0.02) * 10)
let speed9 = abs(sin((frameCount + 180) * 0.04) * 10)
let speed10 = abs(sin((frameCount + 200) * 0.04) * 10)
let speed11 = abs(sin((frameCount + 220) * 0.03) * 10)
let speed12 = abs(sin((frameCount + 240) * 0.02) * 10)
let speed13 = abs(sin((frameCount + 260) * 0.05) * 10)
let speed14 = abs(sin((frameCount + 280) * 0.04) * 10)
```

```
text(word, x0, y0)
x0 += speed0
if(x0 >= width) {
  x0 = 0
}
```

```
text(word, x1, y1)
x1 += speed1
if(x1 >= width) {
  x1 = 0
}
```

```
text(word, x2, y2)
x2 += speed2
if(x2 >= width) {
  x2 = 0
}
```

```
text(word, x3, y3)
x3 += speed3
if(x3 >= width) {
```

```
    x3 = 0
}

text(word, x4, y4)
x4 += speed4
if(x4 >= width) {
    x4 = 0
}

text(word, x5, y5)
x5 += speed5
if(x5 >= width) {
    x5 = 0
}

text(word, x6, y6)
x6 += speed6
if(x6 >= width) {
    x6 = 0
}

text(word, x7, y7)
x7 += speed7
if(x7 >= width) {
    x7 = 0
}

text(word, x8, y8)
x8 += speed8
if(x8 >= width) {
    x8 = 0
}

text(word, x9, y9)
x9 += speed9
if(x9 >= width) {
    x9 = 0
}

text(word, x10, y10)
```

```
x10 += speed10
if(x10 >= width) {
    x10 = 0
}

text(word, x11, y11)
x11 += speed11
if(x11 >= width) {
    x11 = 0
}

text(word, x12, y12)
x12 += speed12
if(x12 >= width) {
    x12 = 0
}

text(word, x13, y13)
x13 += speed13
if(x13 >= width) {
    x13 = 0
}

text(word, x14, y14)
x14 += speed14
if(x14 >= width) {
    x14 = 0
}
}
```



WEDNESDAY

- input by Stephanie on visualising sound
- reading text on females in the electroacoustic music sector:

Übersicht

- Frauen zu Beginn an Entwicklung elektronischer Musik beteiligt; oft sehr einprägend; trotzdem sehr schlecht repräsentiert und größtenteils unerforscht
- Johanna Magdalena Beyer: Pionierin; schrieb eines der ersten Musikstücke, dass nur elektronische Instrumente verwendet
- Eliane Radigue (siehe unten)
- Alice Shields: machte CPEMC bekannter; Performanceszene; Spiel mit Verzerrung von Klavieraufnahmen
- Pril Smiley: erwarb Fähigkeiten am CPEMC; spielte mit Positionen der Klangquellen bei Konzert
- Pauline Oliveros: beteiligt an Gründung des San Francisco Tape Music Center; verzichtet auf traditionelle Instrumente, nutzt Stimme, elektronische Geräte, Synthesizer und Tonband; performative Elemente

- Ruth Anderson: galvanische Klanganzüge; versucht Körperstrom mit Sinusgenerator hörbar zu machen
- Annea Lockwood: verteilte Klangquellen, stellte sich als Performern in die Mitte; erforscht Umgebungsgeräusche und Naturklänge
- Maryanne Anmachen: verbindet und überträgt mit Telefonleitungen Klänge verschiedener Städte; raumgreifende Hörerfahrungen -> Klanginstallationen; Klänge verweben sich

Eliane Radigue

- französische Komponistin
- studierte bei Pierre Schaeffer (Erfinder der Music concrète -> Klangsplitter) & Pierre Henry
- Komposition „Symphonie monotone“ (mit Henry): 20 min Akkord, 20 min Stille
- nach Trennung mit Ehemann Umzug nach Amerika; hier erste reine Synthesizer-basierte Kompositionen
- Slow-Motion-Tracks: aufgenommen mit Magnetbändern und ARP 2500 Modularsystem; langsame, intime, schwebende Klänge
- „Charakteristisch für alle ihre Kompositionen ist die Intimität der leisen und doch ganz gezielt gesetzten Klänge, die Sensibilität für elektronische Sounds und das minimalistisch, reduzierte Klangmaterial, das eine unbeschreibliche Vielzahl von Stimmungen entwickelt“

Music sample: <https://www.youtube.com/watch?v=DCHLgLG22kQ>

- peer teaching

THURSDAY

self-study: working on the code snippets

SPINNING WORDS (SPEED)

```
let word = "SOUND"
// number of words per row
```

```
let wordCount = 20
// number of rows
let rowCount = 8

function setup() {
  createCanvas(windowWidth, windowHeight)
}

function draw() {
  background(0)
  fill(255)
  textSize(30)

  let posX = width / 2
  let posY = height / 2
  let wordWidth = textWidth(word)

  for(let j = 0; j < rowCount; j++) {
    // radius changes for every repetition => creates multiple rows
    let radius = 5 + j * (wordWidth + 10)

    let speed = 0.2 * j
    //let speed = 0.2

    let direction
    // if number of row is even (divisible by two), positive direction
    if(j % 2 === 0) {
      direction = 1
    } // else (meaning row number is odd), negative direction
    } else {
      direction = -1
    }

    // rowOffset calculates offset for rotation:
    // for every frame, number grows; speed can be
    // positive or negative (depending on direction)

    let rotation = 360 / wordCount
```

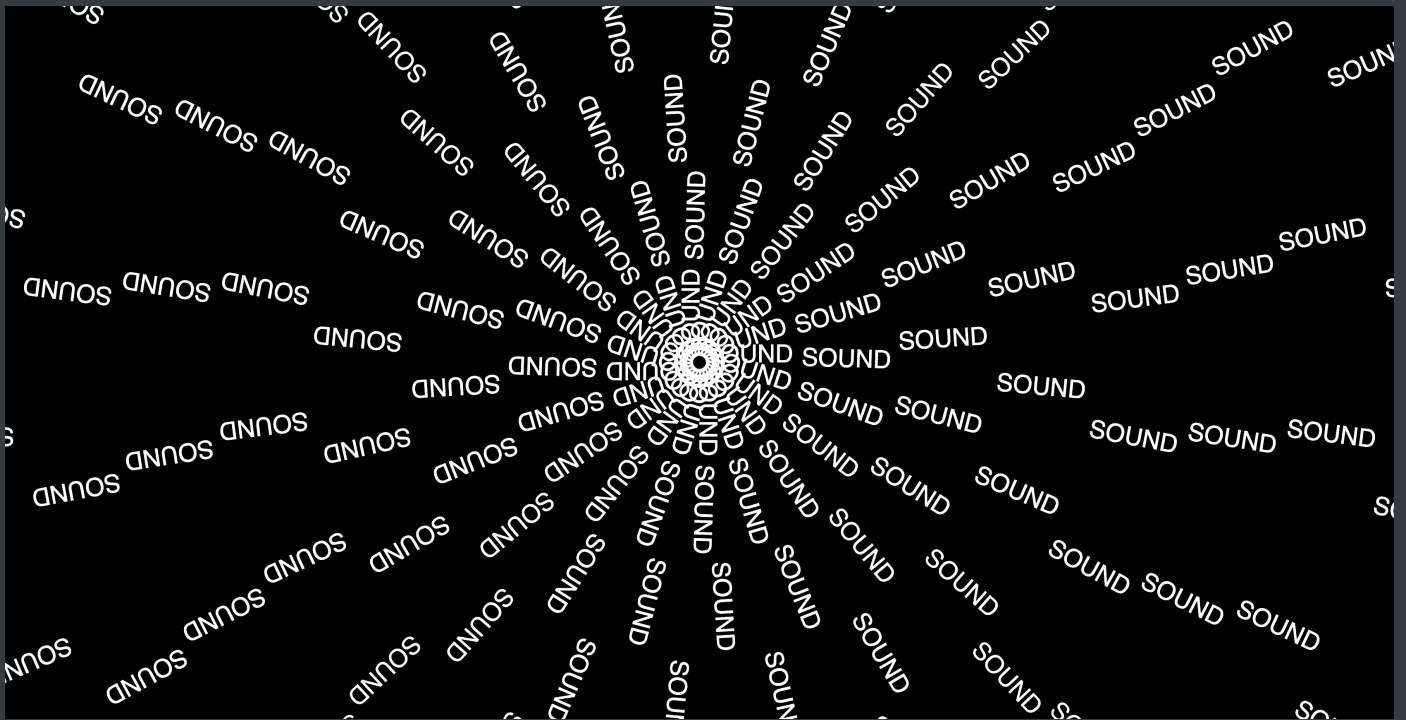


```

for(let i = 0; i < wordCount; i++) {
  let rowOffset = frameCount * speed * direction
  // calculated offset is added to the angle
  let angle = i * rotation + rowOffset
  // calculates cos of angle = x component of the point
  // of angle on circle with radius 1
  // with * radius, it is adapted to the real radius
  // of the row
  let x = posX + cos(radians(angle)) * radius
  // like let x, but sin of angle is calculated =
  // y component of point
  let y = posY + sin(radians(angle)) * radius
  // writes word at position x/y with the angle which
  // changes continuously according to rowOffset
  textRot(word, x, y, angle)
}
}
}

function textRot(txt, x, y, angle) {
  push()
  translate(x, y)
  rotate(radians(angle))
  text(txt, 0, 0)
  pop()
}

```



SPINNING WORDS (VARIABLE SPEEDS)

```
let word = "SOUND"
// number of words per row
let wordCount = 18
// number of rows
let rowCount = 8

function setup() {
  createCanvas(windowWidth, windowHeight)
}

function draw() {
  background(0)
  fill(255)
  textSize(30)

  let posX = width / 2
  let posY = height / 2
  let wordWidth = textWidth(word)
```

```

for(let j = 0; j < rowCount; j++) {
  // radius changes for every repetition => creates multiple rows
  let radius = 5 + j * (wordWidth + 10)
  let speed = 0.2 // base speed
  let direction
  // if number of row is even (divisible by two), positive direction
  if(j % 2 === 0) {
    direction = 1
  // else (meaning row number is odd), negative direction
  } else {
    direction = -1
  }

  // rowOffset calculates offset for rotation:
  // for every frame, number grows; speed can be
  // positive or negative (depending on direction)

  let rotation = 360 / wordCount

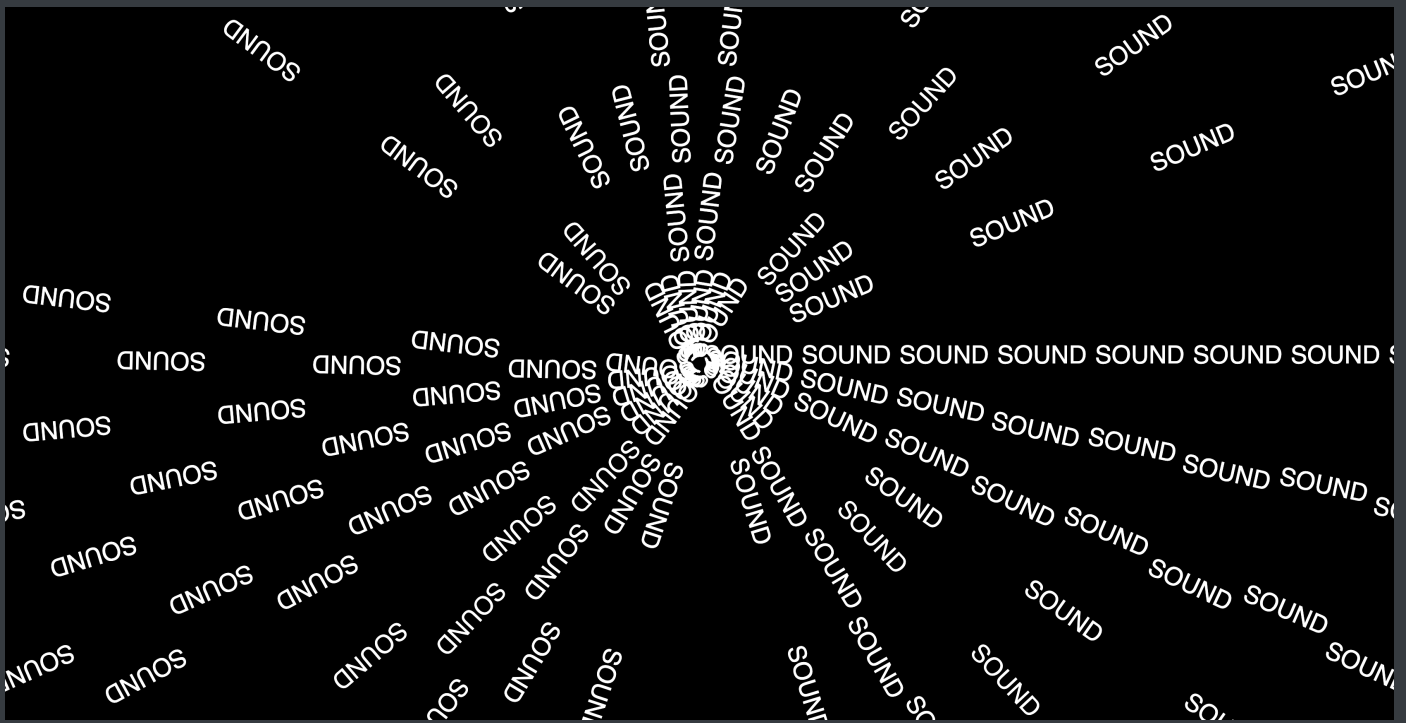
  for(let i = 0; i < wordCount; i++) {

    let rowOffset = frameCount * speed * direction * i * 0.6
    //let rowOffset = frameCount * speed * direction * i/j

    // calculated offset is added to the angle
    let angle = i * rotation + rowOffset
    // calculates cos of angle = x component of the point
    // of angle on circle with radius 1
    // with * radius, it is adapted to the real radius
    // of the row
    let x = posX + cos(radians(angle)) * radius
    // like let x, but sin of angle is calculated =
    // y component of point
    let y = posY + sin(radians(angle)) * radius
    // writes word at position x/y with the angle which
    // changes continuously according to rowOffset
    textRot(word, x, y, angle)
  }
}
}

```

```
function textRot(txt, x, y, angle) {
  push()
  translate(x, y)
  rotate(radians(angle))
  text(txt, 0, 0)
  pop()
}
```



SPINNING WORDS (RADIUS, AUDIO)

```
let word = "SOUND"
// number of words per row
let wordCount = 20
// number of rows
let rowCount = 8

function setup() {
  createCanvas(windowWidth, windowHeight)
  setupAudio(true) // global vars
  // a5.ease = .075 // set easing
}
```

```
}
```

```
function draw() {  
  /* audio vars: amp, ampEase, fft, fftEase, waveform, waveformEase */  
  updateAudio()  
  
  background(0)  
  fill(255)  
  textSize(30)  
  
  let posX = width / 2  
  let posY = height / 2  
  let wordWidth = textWidth(word)  
  
  for(let j = 0; j < rowCount; j++) {  
    // radius changes for every repetition => creates multiple rows  
  
    //let radiusVar = sin(frameCount * 0.2) * 15  
    //let radiusVar = sin(frameCount * 0.2) * 15 * j  
    let radiusVar = sin(frameCount * 0.2) * 3 + 5 * ampEase  
  
    let radius = radiusVar + 5 + j * (wordWidth + 10)  
    let speed = 0.2  
    let direction  
    // if number of row is even (divisible by two), positive direction  
    if(j % 2 === 0) {  
      direction = 1  
      // else (meaning row number is odd), negative direction  
    } else {  
      direction = -1  
    }  
  
    // rowOffset calculates offset for rotation:  
    // for every frame, number grows; speed can be  
    // positive or negative (depending on direction)  
  
    let rotation = 360 / wordCount  
  
    for(let i = 0; i < wordCount; i++) {  
      let rowOffset = frameCount * speed * direction
```

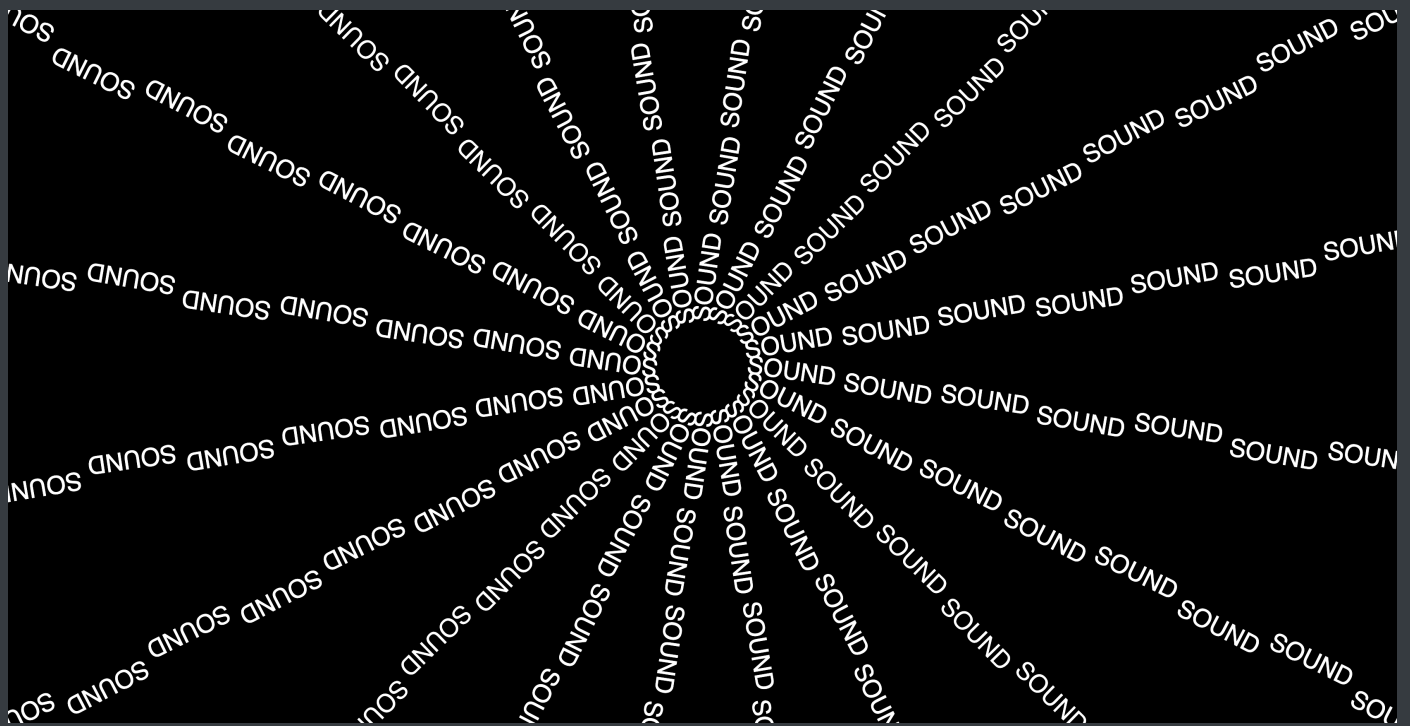
```

        // calculated offset is added to the angle
        let angle = i * rotation + rowOffset
        // calculates cos of angle = x component of the point
        // of angle on circle with radius 1
        // with * radius, it is adapted to the real radius
        // of the row
        let x = posX + cos(radians(angle)) * radius
        // like let x, but sin of angle is calculated =
        // y component of point
        let y = posY + sin(radians(angle)) * radius
        // writes word at position x/y with the angle which
        // changes continuously according to rowOffset
        textRot(word, x, y, angle)
    }
}

}

function textRot(txt, x, y, angle) {
    push()
    translate(x, y)
    rotate(radians(angle))
    text(txt, 0, 0)
    pop()
}

```



SPINNING WORDS (SIN)

```
let word = "SOUND"
// number of words per row
let wordCount = 18
// number of rows
let rowCount = 16

function setup() {
  createCanvas(windowWidth, windowHeight)
}

function draw() {
  background(0)
  fill(255)
  textSize(15)

  let posX = width / 2
  let posY = height / 2
  let wordWidth = textWidth(word)

  for(let j = 0; j < rowCount; j++) {
    // radius changes for every repetition => creates multiple rows
```

```

let radius = 5 + j * (wordWidth + 10)
let speed = 0.2
let direction = sin (frameCount * 0.01) * 0.2
// if number of row is even (divisible by two), positive direction
//if(j % 2 === 0) {
    //direction = 1
    // else (meaning row number is odd), negative direction
//} else {
//  direction = -1
//}

// rowOffset calculates offset for rotation:
// for every frame, number grows; speed can be
// positive or negative (depending on direction)

let rotation = 360 / wordCount

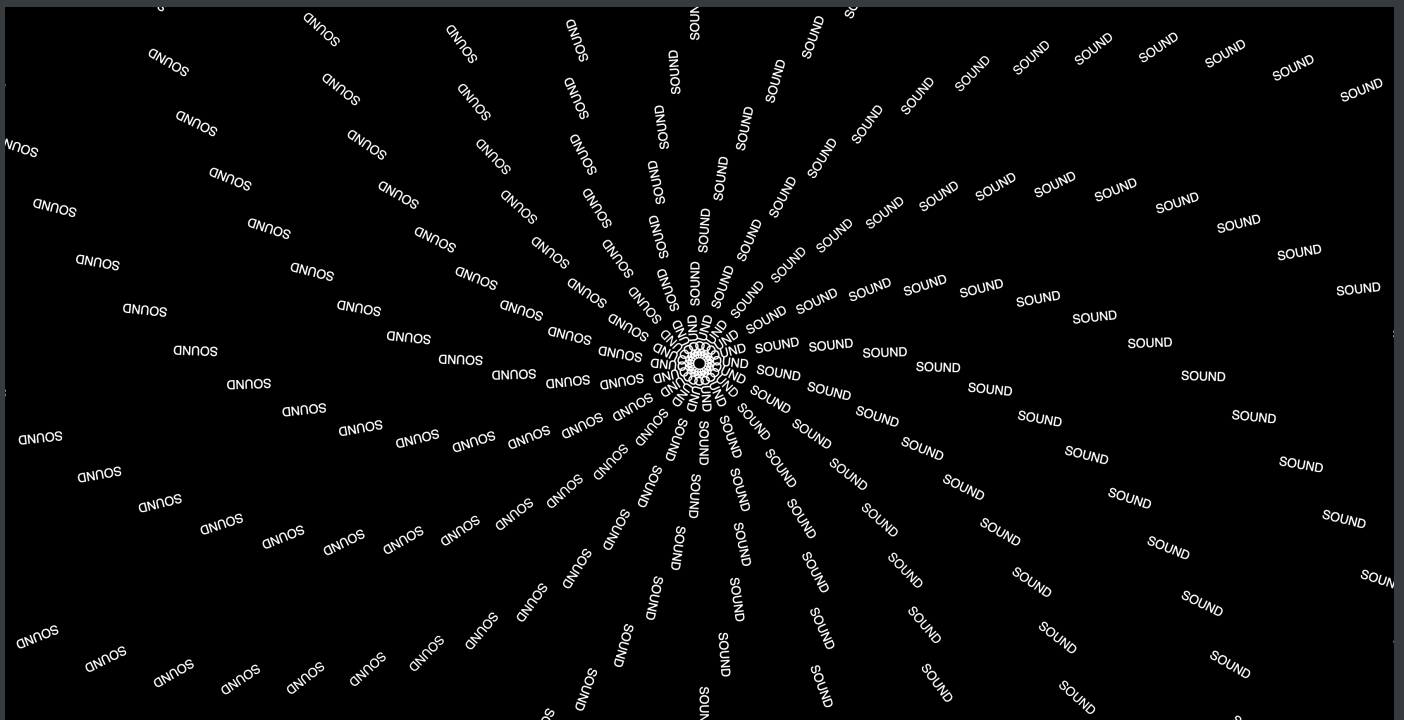
for(let i = 0; i < wordCount; i++) {
    let rowOffset = frameCount * speed * direction
    // words of different rows are offset to each other
    let angleOffset = j * 4

    // calculated offset is added to the angle
    let angle = i * rotation + rowOffset + angleOffset
    // calculates cos of angle = x component of the point
    // of angle on circle with radius 1
    // with * radius, it is adapted to the real radius
    // of the row
    let x = posX + cos(radians(angle)) * radius
    // like let x, but sin of angle is calculated =
    // y component of point
    let y = posY + sin(radians(angle)) * radius
    // writes word at position x/y with the angle which
    // changes continuously according to rowOffset
    textRot(word, x, y, angle)
}
}
}

```



```
function textRot(txt, x, y, angle) {
  push()
  translate(x, y)
  rotate(radians(angle))
  text(txt, 0, 0)
  pop()
}
```



STRETCHED CHARACTERS IN GRID

```
function setup() {
  createCanvas(windowWidth, windowHeight)
  frameRate(3)
}

function draw() {
  background(0)
  let amountX = 12
  let amountY = 8
  let gridX = windowWidth / amountX
  let gridY = windowHeight / amountY
```

```
let letterRep = [1, 2, 3, 4]
let wordArray = ["1", "2", "3", "4"]
let posY = 0
let posX = 0
```

```
textAlign(LEFT, TOP)
```

```
for(let k = 0; k < amountY; k++) {
  for(let j = 0; j < amountX; j++) {
    textSize(gridY * 1.3)
    // frameCount gets higher; random number from 0 to 4 is added; result
    // is calculated down to length of wordArray -> lies between 0 and 3
```

because

```
    // 4 elements in wordArray (0,1,2,3)
    let i = (frameCount + floor(random(4))) % wordArray.length
    let word = wordArray[i]
    let repeatedText = word.repeat(letterRep[i])
    let scaleX = gridX / textWidth(repeatedText)
    fill(255)
    push()
    translate(j * gridX, k * gridY)
    scale(scaleX, 1)
    text(repeatedText, 0, 0)
    pop()
  }
}
```



FRIDAY

morning

- getting to know Yann's work as an artist
- questions of students

afternoon

- repetition of basic functions and commands
- coding with audio in 3d

AUDIOREACTIVE ROTATING SHAPES

```
/*  
  _audio_analysis // cc teddavis.org 2019-24  
  
  revamp of P5LIVE's Audio snippet, now built-in the background!
```

```
add snippet to other sketches: CTRL + SHIFT + A
*/

function setup() {
  createCanvas(windowWidth, windowHeight, WEBGL)

  // audio stuff now behind the scenes, 'true' makes class vars global
  setupAudio(true) // if empty, use 'a5.' before audio vars below
  a5.ease = .275 // customize ease speed (kleiner wert = smooth)
}

function draw() {

  /* audio vars: amp, ampL, ampR, ampEase, fft, fftEase, waveform, waveformEase
  */
  updateAudio()
  lights()

  noFill()
  background(0)

  // green torus
  stroke(126, 255, 1)
  // fftEase[0] -> only analyses the lowest sounds of the audio analysis
  // the higher the number, the higher the sounds that are influencing it
  // fftEase is the smoothened out version of command fft
  trs(50, 50, 30, 50 + fft[20], 2 )

  // pink cube
  stroke(211, 1, 255)
  cube(0, 20, -50, 100 + fftEase[100], 2)

  // orange sphere
  stroke(255, 116, 1)
  sph(-100, 50, 100, 10 + fftEase[60], 3)

}

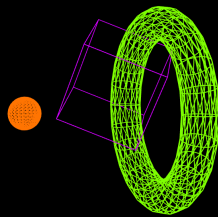
function sph(x,y,z,size, rSpeed) {
```

```
push()
translate(x,y,z)
rotateY(frameCount * 0.01 * rSpeed)
sphere(size)
pop()
}
```

```
function cube(x,y,z,size,rSpeed) {
  push()
  translate(x,y,z)
  rotateZ(frameCount * 0.01 * rSpeed)
  rotateY(frameCount * 0.01 * rSpeed)
  box(size)
  pop()
}
```

```
function trs(x, y, z, size, rSpeed) {
  push()
  translate(x,y,z)
  rotateY(frameCount * 0.01 * rSpeed)
  torus(size, 20)
  pop()
}
```

```
// /* fftEase */
// for(let i = 0; i < fftEase.length; i++) {
//   let freq = fftEase[i]; // (0, 255)
//   let x = map(i, 0, fftEase.length, 0, width)
//   let w = width / fftEase.length
//   rect(x, height * .805, w, freq)
// }
```



AUDIOREACTIVE SHAPES WITH LIGHTING

```
/*
  _audio_analysis // cc teddavis.org 2019-24

  revamp of P5LIVE's Audio snippet, now built-in the background!
  add snippet to other sketches: CTRL + SHIFT + A
*/

function setup() {
  createCanvas(windowWidth, windowHeight, WEBGL)

  // audio stuff now behind the scenes, 'true' makes class vars global
  setupAudio(true) // if empty, use 'a5.' before audio vars below
  a5.ease = .275 // customize ease speed (kleiner wert = smooth)
}

function draw() {

  /* audio vars: amp, ampL, ampR, ampEase, fft, fftEase, waveform, waveformEase
  */
  updateAudio()
  //gives objects a 3d look by adding lights and shadows
}
```

```

lights()
//ambientLight(50)
//gives the light a colour -> pink
//ambientMaterial(255, 0, 255)
// reddish light; first 3 values are colour, rest is point (x,y,z) where it
originates
directionalLight (255,0,0,-10,10,0)
noStroke()
background(0)
//specularMaterial(255)

// green torus
fill(126, 255, 1)
// fftEase[0] -> only analyses the lowest sounds of the audio analysis
// the higher the number, the higher the sounds that are influencing it
// fftEase is the smoothened out version of command fft
trs(50, 50, 30, 50 + fft[20], 2 )

// pink cube
fill(211, 1, 255)
cube(0, 20, -50, 100 + fftEase[100], 2)

// orange sphere
fill(255, 116, 1)
sph(-100, 50, 100, 10 + fftEase[60], 3)

}

function sph(x,y,z,size, rSpeed) {
  push()
  translate(x,y,z)
  rotateY(frameCount * 0.01 * rSpeed)
  sphere(size)
  pop()
}

function cube(x,y,z,size,rSpeed) {
  push()
  translate(x,y,z)

```

```

    rotateZ(frameCount * 0.01 * rSpeed)
    rotateY(frameCount * 0.01 * rSpeed)
    box(size)
    pop()
}

function trs(x, y, z, size, rSpeed) {
    push()
    translate(x,y,z)
    rotateY(frameCount * 0.01 * rSpeed)
    torus(size, 20)
    pop()
}

// /* fftEase */
// for(let i = 0; i < fftEase.length; i++) {
//   let freq = fftEase[i]; // (0, 255)
//   let x = map(i, 0, fftEase.length, 0, width)
//   let w = width / fftEase.length
//   rect(x, height * .805, w, freq)
// }

```



3D AUDIO FOR-LOOP

```
/*
  _audio_analysis // cc teddavis.org 2019-24

  revamp of P5LIVE's Audio snippet, now built-in the background!
  add snippet to other sketches: CTRL + SHIFT + A
*/

function setup() {
  createCanvas(windowWidth, windowHeight, WEBGL)

  // audio stuff now behind the scenes, 'true' makes class vars global
  setupAudio(true) // if empty, use 'a5.' before audio vars below
  a5.ease = .275 // customize ease speed (kleiner wert = smooth)
}

function draw() {
  background(0)
  /* audio vars: amp, ampL, ampR, ampEase, fft, fftEase, waveform, waveformEase
  */
  updateAudio()

  orbitControl()

  // reddish light; first 3 values are colour, rest is point (x,y,z) where it
  originates
  strokeWeight(1)

  //gives objects a 3d look by adding lights and shadows
  lights()
  directionalLight(0, 0, 255, -10, 10, 0)

  let number = 6
  let index = 0

  // modular function % always returns you what remains if the current number
  // is divided by the number after the % character. thus, if we have
  frameCount%4,
```

```

// it will give you 0,1,2,3,0,1,2,3 because every fourth number is dividable
by 4
// and thus there's no rest
// 10%5 = 0; 10%4 = 2 (because 2*4 = 8 + 2 = 10)

//fft.Ease is the array which stores 128 different classes of frequencies;
// fft.Ease [0] gives you the first element in this array => what is stored
// in the lowest frequency class

for(let x = 0; x < number; x++) {
  let posX = map(x, 0, number - 1, -width / 4, width / 4)
  for(let y = 0; y < number; y++) {
    let posY = map(y, 0, number - 1, -width / 4, width / 4)
    for(let z = 0; z < number; z++) {
      let posZ = map(z, 0, number - 1, -width / 4, width / 4)
      if(index % 9 === 0) {
        stroke(0, 0, 255)
        fill(255, 0, 255)
        cube(posX, posY, posZ, 40 + fftEase[index % fftEase.length] * 0.25, 1)
      }
      else if (index % 7 === 0) {
        fill(255, 0, 255)
        cube(posX, posY, posZ, 40 + fftEase[index % fftEase.length] * 0.25, 1)
      }

      else {
        push()
        noStroke()
        fill(255)
        trs(posX, posY, posZ, 40 + fftEase[index % fftEase.length] * 0.25, 1)
        pop()
      }
      index++
    }
  }
}

}

function sph(x, y, z, size, rSpeed) {

```

```
push()
translate(x, y, z)
rotateY(frameCount * 0.01 * rSpeed)
sphere(size)
pop()
}
```

```
function cube(x, y, z, size, rSpeed) {
  push()
  translate(x, y, z)
  rotateZ(frameCount * 0.01 * rSpeed)
  rotateY(frameCount * 0.01 * rSpeed)
  box(size)
  pop()
}
```

```
function trs(x, y, z, size, rSpeed) {
  push()
  translate(x, y, z)
  rotateY(frameCount * 0.01 * rSpeed)
  torus(size, 20, 10, 6)
  pop()
}
```

```
// /* fftEase */
// for(let i = 0; i < fftEase.length; i++) {
//   let freq = fftEase[i]; // (0, 255)
//   let x = map(i, 0, fftEase.length, 0, width)
//   let w = width / fftEase.length
//   rect(x, height * .805, w, freq)
// }
```

